

BIGCOIN · SECURITY REVIEW

Bigcoin Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Bigcoin	Review	Guardian	September 29 – October 22, 2025	October 29, 2025

§ 00

Contents.

30 pages — overview, confidence ranking, scope, 20 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 13 findings	P. 09
	Remediation Review 1 · 5 findings	P. 22
	Remediation Review 2 · 2 findings	P. 27
§ 05	Disclaimer	P. 29
§ 06	About Guardian	P. 30

§ 01

Overview.

Guardian reviewed Review across 3 review rounds. The final report includes 1 Critical, 4 High, 3 Medium, 5 Low, 7 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	bigcoinmining/bigcoin-contracts
CLIENT	Bigcoin	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2
PROJECT	Review	MAIN REVIEW COMMIT	b74559a2bc4e
PREPARED BY	Guardian	REMEDATION REVIEW 1 COMMIT	1df2ee2372d4
AUDIT WINDOW	September 29 – October 22, 2025	REMEDATION REVIEW 2 COMMIT	23e2fda2ad33
FINAL REPORT	October 29, 2025		

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	1	0	0	0	1
■ High	4	0	1	0	3
■ Medium	3	0	0	0	3
■ Low	5	0	0	0	5
■ Informational	7	0	5	0	2
All severities	20	0	6	0	14

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `bigcoinmining/bigcoin-contracts`. The scope includes 1,701 source lines across 12 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/MainV2.sol	738	1,247
src/Types.sol	63	87
src/minepad/IMinepad.sol	3	5
src/minepad/IMinepadFactory.sol	3	5
src/libraries/Errors.sol	44	46
src/libraries/Events.sol	47	77
src/minepad/uniswap-v3/LiquidityLocker.sol	120	201
src/minepad/uniswap-v3/LiquidityLockerFactory.sol	39	81
src/minepad/uniswap-v3/MinepadToken.sol	241	356
src/minepad/uniswap-v3/MinepadTokenFactory.sol	211	375
src/minepad/uniswap-v3/TickMath.sol	168	205
src/minepad/uniswap-v3/Types.sol	24	30

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Sniping Risk Via Liquidity Cap Finalization	Logical Error	CRITICAL	RESOLVED	p. 9
H-01	supplyForMiners Instead Of supplyForLPs	Logical Error	HIGH	RESOLVED	p. 10
H-02	No Retry Path On Migration Failiure	Logical Error	HIGH	RESOLVED	p. 11
H-03	removeLPGuardLq Reverts	DoS	HIGH	ACKNOWLEDGED	p. 12
M-01	Incorrect Address Used In _canBuyMiner	Logical Error	MEDIUM	RESOLVED	p. 13
M-02	deployMinepad Misuse	Validation	MEDIUM	RESOLVED	p. 14
L-01	minepadCompleted Uses Wrong Address	Logical Error	LOW	RESOLVED	p. 15
L-02	syncMinepad Can Trigger Before Allocation	Frontrunning	LOW	RESOLVED	p. 16
I-01	Remainder Left In Minepad Contract	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-02	Referral Fee Not Applied With Minepad	Warning	INFORMATIONAL	RESOLVED	p. 18
I-03	Predictable Address Enables Pool Griefing	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 19
I-04	_mint Function Reverts Past Cap	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 20
I-05	Inconsistent TickMath Version	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 21

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDIATION REVIEW 1					
H-01	Hardcoded Swap Direction Can Block Migration	DoS	HIGH	RESOLVED	p. 22
L-01	Overwritten _extraForLpRebalance Value	Logical Error	LOW	RESOLVED	p. 23
L-02	LP Rebalance Not Deducted	Logical Error	LOW	RESOLVED	p. 24
L-03	Retry Migration Enables Sandwich Attack	Access Control	LOW	RESOLVED	p. 25
I-01	Guarded Liquidity Constrained	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 26
REMEDIATION REVIEW 2					
M-01	Inverted Amount In _migrateLiquidity	Logical Error	MEDIUM	RESOLVED	p. 27
I-01	Unbounded Creator Token Allocation	Informational	INFORMATIONAL	RESOLVED	p. 28

CRITICAL **C-01**

RESOLVED

Sniping Risk Via Liquidity Cap Finalization

CATEGORY	Logical Error
LOCATION	src/minepad/uniswap-v3/MinepadToken.sol#L216-L217
REVIEW	Main Review

DESCRIPTION

The allocated hashrate mechanism in Minepad is designed to give all participants enough time to gradually obtain token allocations and discourage sniping. However, because mining can also be finalized once the liquidity cap is satisfied, a sniping attack remains possible.

Attack scenario:

1. An attacker allows the creator to deploy the token.
2. They allocate some hash rate.
3. They donate Bigcoin such that the `balanceOf` is greater than the cap.
4. They then trigger finalization.

This would allow them to receive the entire miners' supply of Minepad tokens in exchange for the Bigcoin that went into LP. They could later recover those funds by continuing to mine.

So instead of waiting for Bigcoin rewards to accrue before Minepad tokens can finalize, they can do it instantly.

And if the attacker is also the creator, they don't even lose their donated Bigcoin, since they would get it back after vesting.

This behavior undermines the anti-sniping intent of the system and introduces a loophole.

RECOMMENDATION

Consider tracking the bigcoin rewards sent to minepad token instead of relying on `balanceOf` alone.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/commit/eded2a497e02f40c75cdb4712802d8d5dc55320b>

HIGH **H-01**

RESOLVED

supplyForMiners Instead Of supplyForLPs

CATEGORY	Logical Error
LOCATION	src/minepad/uniswap-v3/MinepadToken.sol#L283-L284
REVIEW	Main Review

DESCRIPTION

During finalization, if `sqrtPriceX96Start != sqrtPriceX96`, a swap is attempted to reach the desired price.

If there is zero liquidity in the pool, this swap occurs without pulling tokens from the Minepad token. However, if anyone has already added liquidity to the pool directly, the Minepad token handles this case by rebalancing through `extraForLpRebalance`.

The problem is that this rebalancing amount is **subtracted from `supplyForMiners` instead of `supplyForLPs` **. Since the swap pulls `extraForLpRebalance` worth of Minepad tokens and these tokens accrue to LPs, they should be subtracted from `supplyForLPs`. Otherwise, it dilutes or even eliminates the allocation to miners.

In the worst case, if `extraForLpRebalance >= supplyForMiners`, miners would receive no allocation at all, and all rewards would go to users who provided liquidity directly on Uniswap v3.

RECOMMENDATION

Consider reducing `extraLPForRebalance` from the LP balance instead of the miners' balance. However, there could still be an underflow problem if direct liquidity addition on Uniswap v3 is too large. We understand that this scenario is not profitable, but ideally, there should not be any edge case like that. Please give us some time to come back with a finalized solution.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/10/commits/896ac960080c3e3e8e46e77a2fa1917e8b2096a7>

HIGH **H-02**

RESOLVED

No Retry Path On Migration Failiure

CATEGORY	Logical Error
LOCATION	src/minepad/uniswap-v3/MinepadToken.sol#L223
REVIEW	Main Review

— DESCRIPTION

In `_finalizeMining()`, the contract first sets `miningCompleted = true`, calls `MAIN.deallocateHashrate()`, and removes the LP guard. If the subsequent finalization steps fail, for example if `_calculatePrice` returns an invalid price or the pool cannot be set exactly at the target `sqrtPriceX96`, the function emits `MigrationFailed` and returns early. At that point, the Minepad is marked as finalized (`miningCompleted = true`), but no LP is minted or locked and no excess amounts paid out. This leaves the Bigcoin and MinepadToken balances intended to seed liquidity stranded in the contract. The creator receives neither LP nor the excess Bigcoin or MinepadToken payouts, and the contract has no mechanism to retry migration or recover the stuck assets.

— RECOMMENDATION

Defer setting `miningCompleted` to true until after a successful LP mint, or introduce a `retryFinalize` mechanism. As a fallback, consider adding a recovery path to sweep stranded Bigcoin or MinepadToken if migration cannot be completed.

— RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/10/commits/f643f5c2244c9602972bb7bc2570f56464108987>

HIGH **H-03**

ACKNOWLEDGED

removeLPGuardLq Reverts

CATEGORY	DoS
LOCATION	src/minepad/uniswap-v3/MinepadTokenFactory.sol#L350
REVIEW	Main Review

— DESCRIPTION

The removeLPGuardLiquidity function removes the LP guard, collects any accrued fees, and burns the position. The issue is that when calling collect, amount0Max and amount1Max are both set to tokensOwed0 and tokensOwed1. If both values are zero, the collect call to UNISWAP_POSITION_MANAGER will revert, since it requires at least one of the amounts to be greater than zero.

As a result, the _finalizeMining function, which calls removeLPGuardLiquidity, will also revert blocking the migration process entirely. This situation can occur if the current price and ticks produce a withdrawal of zero and there are no fees to collect, and can also be exploited by manipulating conditions to effectively prevent finalization.

— RECOMMENDATION

Modify the collect logic in the removeLPGuardLiquidity function to only perform the collect call when either tokensOwed0 or tokensOwed1 is greater than zero.

PROOF OF CONCEPT <https://github.com/GuardianOrg/bigcoin-contracts-fuzz-1758722778293/blob/698cf556c49f848775b7aacf9e786809c70e387b/test/fuzzing/FoundryPlayground.sol#L17>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Incorrect Address Used In _canBuyMiner

CATEGORY Logical Error
LOCATION src/MainV2.sol#L1209
REVIEW Main Review

DESCRIPTION

The canBuyMiner function allows anyone to check whether a facility at a given facilityAddress can buy a miner at a specified location using a miner index and coordinates. The problem is that the _isInvalidCoordinates function, which is called within _canBuyMiner, always uses msg.sender as the facilityAddress. As a result, when a user calls canBuyMiner for a facilityAddress that is not the caller, the coordinate validation is performed against the wrong address. This leads to incorrect results, as the function does not evaluate the intended facility's validity but instead checks coordinates against the caller.

RECOMMENDATION

Modify _isInvalidCoordinates to use a provided facilityAddress argument rather than msg.sender to ensure coordinate validation is performed against the correct facility.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/10/commits/969345a20ae0b7e129820596ccde7789c85d99c3>

MEDIUM **M-02**

RESOLVED

deployMinepad Misuse

CATEGORY Validation

LOCATION src/minepad/uniswap-v3/MinepadTokenFactory.sol#L126

REVIEW Main Review

DESCRIPTION

The `createMinepad` function requires users to approve the `MinepadTokenFactory` for at least the `deploymentFee` in Bigcoin before it can be called. This fee is transferred in `deployMinepad` to the `MinepadTokenFactory` owner address.

A potential issue arises because `deployMinepad` itself is publicly callable and allows specifying an arbitrary `creator` address. If a user has granted the factory the maximum approval or has a lingering approval, an attacker can repeatedly call `deployMinepad` with that user's address as the `creator`, draining the user's entire approved Bigcoin balance. The resulting Minepads would be unusable, since they are not deployed through the intended `createMinepad` flow, but the user's funds would already be consumed.

RECOMMENDATION

Restrict `deployMinepad` so it can only be called from the MainV2 contract, preventing attackers from directly exploiting user approvals.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/10/commits/399fadbd49e674d870a5a945563dc7a1005c1b6d>

LOW **L-01**

RESOLVED

minepadCompleted Uses Wrong Address

CATEGORY Logical Error
LOCATION src/MainV2.sol#L663
REVIEW Main Review

DESCRIPTION

The minepadCompleted view function is intended to take a minepad address as input and check whether it is still active by reading from the minepadDetails mapping. However, the function currently references msg.sender instead of the provided minepad address. As a result, it does not return the correct status of the target minepad and instead always resolves to the default boolean value. This breaks the intended functionality of the function and prevents users or integrators from correctly querying a minepad's completion state.

RECOMMENDATION

Update the function to reference the minepad address passed as input rather than msg.sender when looking up minepadDetails.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/10/commits/b1f9737371647837312cd4ee97344927d590bcf2>

LOW **L-02**

RESOLVED

syncMinepad Can Trigger Before Allocation

CATEGORY Frontrunning
LOCATION src/minepad/uniswap-v3/MinepadToken.sol#L143-L144
REVIEW Main Review

DESCRIPTION

Currently, anyone can call `hashrateUpdated` through `syncMinepad`. Although this scenario is unlikely, if a user has not yet allocated hashrate and someone calls `syncMinepad`, it would start the clock for `miningEndTime` from that sync call itself:

```
miningStartTime = block.timestamp;  
miningEndTime += timeShift;
```

This goes against the protection logic Bitcoin is trying to enforce.

RECOMMENDATION

Consider allowing `syncMinepad` only if mining has already started, or otherwise handle this edge case to avoid unintended early mining timers.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/10/commits/3025c5e5534ba514870b54044fdb251abbd536ad>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Remainder Left In Minepad Contract

CATEGORY	Warning
LOCATION	src/minepad/uniswap-v3/MinepadToken.sol#L386-L387
REVIEW	Main Review

DESCRIPTION

Since `distribute` rounds down when calculating miner rewards:

```
uint256 totalRewards = FixedPointMathLib.mulDiv(
    facilityShares,
    amountForMiners,
    totalShares
); // @audit precision loss
```

a small portion of `amountForMiners` (dust) will remain in the contract.

RECOMMENDATION

Be aware of this behavior. No fix is required. Rounding down is correct direction.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02**

RESOLVED

Referral Fee Not Applied With Minepad

CATEGORY Warning
LOCATION src/MainV2.sol#L605-L606
REVIEW Main Review

DESCRIPTION

For normal claims, Bigcoin takes a referral fee or credits it to the referrer. This is not applied to claims made through Minepad. As a result, Bigcoin generated by hashrate allocated to Minepad does not include a referral fee.

RECOMMENDATION

Review this behavior. If this was not intended, consider fixing it.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/10/commits/8419ccdb66fbe4468641e0b4a82aa51086278b28>

INFORMATIONAL **I-03**

ACKNOWLEDGED

Predictable Address Enables Pool Griefing

CATEGORY	DoS
LOCATION	src/minepad/uniswap-v3/MinepadToken.sol#L133
REVIEW	Main Review

— DESCRIPTION

The MinepadToken constructor reverts with PoolAlreadyCreated() if a Uniswap V3 pool for (MinepadToken, BIGCOIN, feeTier) already exists. Because the Minepad token address is deterministically derived via CREATE2 from the factory's salt loop and deployment parameters, an attacker can precompute the future token address and front-run deployment. By calling Uniswap V3 createPool for that pair before the constructor executes, the attacker can create a pool against the future address. Uniswap V3 permits permissionless pool creation for arbitrary addresses (even if no code is deployed yet), so this succeeds. When the creator later calls createMinepad, the constructor detects the preexisting pool and reverts, blocking deployment.

— RECOMMENDATION

Be aware of this griefing vector that can block deployments; affected creators can typically resolve it by retrying once the factory's salt advances, yielding a new MinepadToken address.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

_mint Function Reverts Past Cap

CATEGORY Informational
LOCATION src/MainV2.sol#L819
REVIEW Main Review

— DESCRIPTION

The `_mint` function in the MainV2 contract enforces a supply cap by reverting if `amtBurned + supply + amount` exceeds `MAX_SUPPLY + halvingOvershoot`. Once this limit is reached, all functions that call `_mint` will revert.

— RECOMMENDATION

If this behaviour is intended, ensure it does not unintentionally block any critical functionality that it's not supposed to.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

ACKNOWLEDGED

Inconsistent TickMath Version

CATEGORY	Informational
LOCATION	library.TickMath
REVIEW	Main Review

— DESCRIPTION

The TickMath library specifies `pragma solidity ^0.8.19`, while the Uniswap V3 TickMath library uses `pragma solidity >=0.5.0 <0.8.0`. Although this currently has no observable impact, it could introduce subtle issues if any parts of the codebase rely on underflow or overflow behaviour that was possible in pre-0.8 Solidity versions.

— RECOMMENDATION

Consider migrating to the Uniswap V4 TickMath implementation to ensure compatibility with Solidity 0.8's built-in arithmetic safety and prevent unintended behaviour in edge-case calculations.

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

RESOLVED

Hardcoded Swap Direction Can Block Migration

CATEGORY DoS
LOCATION src/minepad/uniswap-v3/MinepadToken.sol#L337
REVIEW Remediation Review 1

DESCRIPTION

The `_migrateLiquidity` function attempts to move the pool price to match the target before minting the position. However, the internal swap direction is hardcoded to `zeroForOne = false`. This creates a scenario where, if a user performs a swap between minepad creation and migration, they can remove the Minepad tokens seeded by the LP guard and push the pool price above the target. Once this happens, the migration's price adjustment logic will always fail to bring the price back down, resulting in a blocked migration and preventing LP creation.

RECOMMENDATION

Consider dynamically determining the swap direction in `_adjustPoolPrice` based on the current pool price relative to the target to prevent migration blockage.

PROOF OF CONCEPT <https://github.com/Guardian0rg/bigcoin-contracts-team1-1758722773151/blob/f114808d938f62a3421f2a253cbb73f4d0d07e14/test/minepad/Minepad.t.sol#L551>

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/7/commits/27ef3f8f6138db83d6578f512e450d4def37bb0d>

LOW **L-01**

RESOLVED

Overwritten `_extraForLpRebalance` Value

CATEGORY	Logical Error
LOCATION	src/minepad/uniswap-v3/MinepadToken.sol#L357
REVIEW	Remediation Review 1

DESCRIPTION

The `uniswapV3SwapCallback` function assigns `_extraForLpRebalance` to `amount0Delta`, representing the additional tokens used for LP rebalancing during migration. This value is later deducted from `supplyForLP` in `_migrateLiquidity`. However, `_extraForLpRebalance` is overwritten each time `uniswapV3SwapCallback` is invoked, rather than accumulated. Because this callback can be triggered multiple times, such as during migration retries, only the most recent `amount0Delta` is reflected, leading to an inaccurate LP supply deduction.

RECOMMENDATION

Accumulate `_extraForLpRebalance` across all swap callback calls rather than overwriting it, ensuring the total extra tokens used for LP rebalancing are accurately reflected during migration.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/7/commits/7aadd2d8022d6f7717664ae027756e31e11f1cbd>

LOW **L-02**

RESOLVED

LP Rebalance Not Deducted

CATEGORY Logical Error
LOCATION src/minepad/uniswap-v3/MinepadToken.sol#L194
REVIEW Remediation Review 1

DESCRIPTION

The `withdrawForManualMigration` function allows the protocol owner to manually withdraw liquidity and BIGCOIN from the contract to create a new LP if the migration process fails. However, unlike the successful migration path, this function does not deduct `_extraForLpRebalance` from `supplyForLP`. As a result, the missing deduction is effectively taken from `supplyForMiners`.

RECOMMENDATION

Ensure `_extraForLpRebalance` is deducted from `supplyForLP` during manual migrations to maintain consistent internal accounting.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/7/commits/b7390242b74aa0dbb9dc9e46ccfe8eff8cec74f4>

LOW **L-03**

RESOLVED

Retry Migration Enables Sandwich Attack

CATEGORY	Access Control
LOCATION	src/minepad/uniswap-v3/MinepadToken.sol#L183-L184
REVIEW	Remediation Review 1

DESCRIPTION

The Retry Migration function currently allows anyone to trigger a retry after the first migration failure. Since the Minepad token swap is predetermined, this opens an attack vector for predictable sandwiching, where an attacker can manipulate token prices or extract value from the transaction.

Although the likelihood of a retry migration occurring is relatively low, and the post-migration check

```
sqrtPriceX96Start = sqrtPriceX96
```

ensures that the intended Minepad state is achieved, the open access still poses a risk. Attackers can potentially sandwich the excess Bitcoin or Minepad tokens that should have been burned or distributed to miners, leading to unintended value leakage.

RECOMMENDATION

Consider implementing access control for the Retry Migration function.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/7/commits/a65d30db905c2345f006696d5b654e9d65a57070>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Guarded Liquidity Constrained

CATEGORY	Configuration
LOCATION	src/minepad/uniswap-v3/MinepadTokenFactory.sol#L384-L385
REVIEW	Remediation Review 1

DESCRIPTION

The current Bigcoin implementation restricts guarded liquidity to a single tick spacing using the following logic:

```
_constrainedTickLower = MIN_CONSTRAINED_TICK_LOWER;  
_constrainedTickLower = (_constrainedTickLower / _tickSpacing) * _tickSpacing;  
_constrainedTickUpper = _constrainedTickLower + _tickSpacing;
```

Because liquidity is only allocated within one tick, it becomes possible for an attacker or trader to remove all guarded liquidity (1 minepad) with a single swap before finalize with minimal costs. This ensures that any swaps performed before finalization do not consume the entire guarded liquidity in one step, improving liquidity resilience and reducing manipulation risk.

RECOMMENDATION

Consier allocating liquidity across the full range, from `MIN_CONSTRAINED_TICK_LOWER` to `MAX_CONSTRAINED_TICK_LOWER`, similar to a v2-style wide-range position.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Inverted Amount In `_migrateLiquidity`

CATEGORY Logical Error
LOCATION `src/minepad/uniswap-v3/MinepadToken.sol#L323`
REVIEW Remediation Review 2

DESCRIPTION

The `_migrateLiquidity` function sets `zeroForOne` to true when `sqrtPriceX96Start > sqrtPriceX96`, and false otherwise. While this directional logic is correct, the amount selection is inverted. The function uses `bigcoinBalance` when `zeroForOne` is true, and `minepadToken` when false. This is incorrect as the `zeroForOne` swap should use the `token0` amount, which in this case corresponds to the Minepad token. As a result, the swap executes with the wrong `amountSpecified`.

RECOMMENDATION

Ensure the amount aligns with the correct token side for the determined direction in `_migrateLiquidity`, using the Minepad token when `zeroForOne` is true and Bigcoin when false.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/12/commits/0bc92d1cbe7cd14804101942577e7cbab1baf6d>

INFORMATIONAL **I-01**

RESOLVED

Unbounded Creator Token Allocation

CATEGORY	Informational
LOCATION	src/minepad/uniswap-v3/MinepadTokenFactory.sol#L128
REVIEW	Remediation Review 2

DESCRIPTION

The Minepad token contract allows the creator to optionally specify a creator token allocation when initializing a new Minepad. However, there is currently no maximum limit imposed on this allocation. As a result, a creator could assign an excessively large portion of the total supply to themselves.

RECOMMENDATION

Consider setting a maximum cap on the creator token allocation to prevent excessive self-allocation.

RESOLUTION

Status: Resolved.

<https://github.com/bigcoinmining/bigcoin-contracts/pull/12/commits/bfdb81e790be0e3f1960a104e0236bfef7c23731>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.